```
            try:
                sock.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass
            try:
                sock.close()
            except OSError:
                pass
        if self._thread and self._thread.is_alive():
            self._thread.join(timeout=3)

    # ------------------------------------------------------------------
    def _make_server_socket(self):
        """Create and bind the TCP server socket (platform-aware)."""
        server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        # On Windows SO_REUSEADDR is dangerous (lets other processes steal the
        # port). Use SO_EXCLUSIVEADDRUSE instead so only one listener owns it.
        if hasattr(socket, "SO_EXCLUSIVEADDRUSE"):          # Windows
            server.setsockopt(socket.SOL_SOCKET, socket.SO_EXCLUSIVEADDRUSE, 1)
        else:                                               # Linux / macOS
            server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server.bind((self.host, self.port))
        server.listen(5)
        server.settimeout(1.0)
        return server

    # ------------------------------------------------------------------
    def _run(self):
        while not self._stop_event.is_set():
            server = None
            try:
                server = self._make_server_socket()
                with self._socket_lock:
                    self._server_socket = server
                with self._status_lock:
                    self._bind_error = None   # cleared ? bind succeeded
                print(
                    f"[PingStation] Listening on {self.host}:{self.port} "
                    f"? waiting for PingStation connection ?"
                )

                while not self._stop_event.is_set():
                    # --- accept loop ----------------------------------------
                    try:
                        conn, addr = server.accept()
                    except socket.timeout:
                        continue
                    except OSError:
                        if self._stop_event.is_set():
                            break
                        raise

                    with self._socket_lock:
                        self._client_socket = conn

                    with self._status_lock:
                        self._is_connected = True
                        self._total_connections += 1
                    print(
                        f"[PingStation] Client connected from {addr[0]}:{addr[1]}"
                    )

                    try:
                        self._handle_connection(conn, addr)
                    finally:
                        with self._socket_lock:
                            self._client_socket = None
                        with self._status_lock:
                            self._is_connected = False
                        print(f"[PingStation] Client {addr[0]}:{addr[1]} disconnected")

            except OSError as exc:
                if self._stop_event.is_set():
```